Rapport explicatif détaillé — TIPE CPGE 2024

Une approche mathématique pour résoudre
le casse-tête Hashiwokakero

Hafsa MEHDAOUI
CPGE MP* · 2024

Résumé

Ce rapport explique, de façon détaillée et autonome, la partie **technique** du TIPE réalisé en CPGE en 2024 sur le puzzle **Hashiwokakero**. Il s'appuie sur la soutenance PowerPoint, le notebook `M17.ipynb` (Pyomo) et la fiche MCOT.

Le fil conducteur est le suivant : (i) présentation du jeu et de sa complexité NP-complète ; (ii) modélisation en théorie des graphes ; (iii) formulation en programme mixte en nombres entiers (MIP) ; (iv) limites du parcours en profondeur (DFS) ; (v) algorithme Branch-and-Cut ; (vi) comparaison expérimentale des solveurs CBC, GLPK et Ipopt sur trois instances (6×6 et 12×12).

Périmètre. La partie « application à l'aménagement urbain / PAU » présente dans la soutenance n'est **pas** traitée dans ce document.

Auteure : Hafsa MEHDAOUI **Année :** 2024 **Mots-clés :** Hashiwokakero, graphes, DFS, Branch-and-Cut, MIP, CBC, GLPK, Ipopt.

Table des matières

1 Contexte et objectifs du TIPE **3**

1.1 Motivation . . . 3

1.2 Positionnement thématique (MCOT) . . . 3

1.3 Problématique . . . 3

1.4 Objectifs techniques retenus . . . 3

1.5 Hors périmètre de ce rapport . . . 3

2 Le puzzle Hashiwokakero **3**

2.1 Règles du jeu . . . 3

2.2 Complexité . . . 4

2.3 Lien avec la théorie des graphes . . . 4

3 Modélisation par la théorie des graphes **4**

3.1 Graphe non orienté associé . . . 4

3.2 Données d'entrée . . . 4

4 Formulation MIP (Mixed Integer Programming) 5

4.1 Variables de décision . . . 5
4.2 Contrainte de degré (C1) . . . 5
4.3 Contrainte de non-croisement (C2) . . . 5
4.4 Couplage entre X et U (C3 et C3B) . . . 5
4.5 Contraintes de connexité par flot (C4A, C4B) . . . 5
4.6 Fonction objectif . . . 6
4.7 Chaîne d'implémentation (notebook) . . . 6

5 Depth First Search (DFS) 6

5.1 Principe . . . 6
5.2 Intérêt pour Hashi . . . 6
5.3 Limites . . . 6

6 Algorithme Branch-and-Cut 7

6.1 Idée générale . . . 7
6.2 Pourquoi c'est adapté à Hashi . . . 7
6.3 Lien avec les solveurs . . . 7

7 Solveurs utilisés 7

7.1 CBC . . . 7
7.2 GLPK . . . 8
7.3 Ipopt . . . 8
7.4 Protocole expérimental . . . 8

8 Simulations et résultats 8

8.1 Instances étudiées . . . 8
8.2 Tableau de performances . . . 9
8.3 Analyse détaillée . . . 9

9 Synthèse méthodologique 9

10 Conclusion 10

11 Fichiers associés 10

A Rappel des contraintes du modèle (vue code) 10

1 Contexte et objectifs du TIPE

1.1 Motivation

De nombreux jeux de réflexion ont des règles simples, mais leur résolution exige une logique rigoureuse. Hashiwokakero (souvent abrégé en *Hashi*) appartient à cette famille : accessible à un joueur humain, il pose néanmoins un vrai défi algorithmique dès que la grille grossit.

Historiquement, Hashi a été créé par Nikoli, éditeur japonais spécialisé dans les jeux de logique. Le premier puzzle est paru en septembre 1990. Au-delà du divertissement, le jeu a été utilisé comme support pédagogique pour introduire des notions de **combinatoire** et de **théorie des graphes**.

1.2 Positionnement thématique (MCOT)

D'après la fiche MCOT, le TIPE se situe à l'intersection de :

— l'**informatique pratique** (implémentation, simulation, mesure de performances) ;

— l'**informatique théorique** (complexité, graphes, algorithmes) ;

— les **mathématiques** (modélisation, optimisation).

1.3 Problématique

> Comment les techniques d'optimisation algorithmiques comme **Branch-and-Cut** peuvent-elles améliorer l'efficacité de la résolution du puzzle Hashiwokakero ?

Autrement dit : peut-on faire mieux qu'une recherche « naïve » en profondeur, en s'appuyant sur une formulation mathématique et sur des solveurs d'optimisation ?

1.4 Objectifs techniques retenus

1. Développer une **modélisation** de Hashiwokakero via la théorie des graphes.
2. Présenter l'algorithme **Branch-and-Cut** pour dépasser les limites du **DFS**.
3. Établir une **simulation informatique** (Pyomo + solveurs) et comparer les performances (objectif et temps d'exécution).

1.5 Hors périmètre de ce rapport

La soutenance comportait une section « application du jeu à des problèmes réels » (aménagement urbain, embouteillages, plans PAU, etc.). Cette section **n'est pas développée** ici, afin de concentrer le document sur le cœur scientifique et informatique du TIPE (modèle, algorithmes, expériences).

2 Le puzzle Hashiwokakero

2.1 Règles du jeu

On dispose d'une grille rectangulaire. Certaines cases contiennent un cercle appelé **île**, numéroté de 1 à 8. Les autres cases sont vides. Le joueur doit relier les îles par des **ponts** en respectant les règles suivantes :

1. Un pont relie deux îles distinctes.
2. Les ponts sont uniquement **horizontaux** ou **verticaux** (pas de diagonales).
3. Entre deux îles, on peut placer **au plus deux** ponts (simple ou double).
4. Les ponts **ne se croisent pas** et ne traversent aucune autre île.
5. Pour chaque île de valeur v, exactement v ponts touchent cette île.
6. Le graphe obtenu doit être **connexe** : il existe un chemin (via les ponts) entre toute paire d'îles.

Une configuration qui respecte toutes ces règles est une **solution réalisable**.

2.2 Complexité

Andersson (2009) a montré que Hashiwokakero est un problème **NP-complet**. En pratique, cela signifie qu'il n'existe pas (sauf si P = NP) d'algorithme polynomial connu garantissant la résolution optimale de toutes les instances. D'où l'intérêt :

— de bien **modéliser** le problème ;

— d'utiliser des méthodes d'**optimisation combinatoire** (Branch-and-Cut, solveurs MIP) ;

— de **comparer** expérimentalement ces méthodes.

2.3 Lien avec la théorie des graphes

Chaque île devient un **nœud** ; chaque pont possible devient une **arête**. Les contraintes du jeu se traduisent alors en contraintes sur les degrés, l'absence de croisement, et la connexité du graphe. C'est cette traduction qui ouvre la voie à une formulation MIP.

3 Modélisation par la théorie des graphes

3.1 Graphe non orienté associé

Le puzzle est défini sur un graphe non orienté

$$G = (V, E),$$

où :

— $V = \{1, \ldots, n\}$ est l'ensemble des sommets (îles) ;

— E est l'ensemble des arêtes candidates reliant deux îles **adjacentes** : même abscisse ou même ordonnée (alignement horizontal ou vertical).

Dans le notebook, on construit d'abord l'ensemble des couples (c_1, c_2) alignés, noté `lines` (et une version symétrique `ALL_lines` utile pour le flot).

3.2 Données d'entrée

Une instance est fournie sous forme de grille (fichier CSV dans le notebook). Pour chaque case non nulle :

— on enregistre les coordonnées (x, y) de l'île ;

— on enregistre la valeur v (chiffre de 1 à 8).

On obtient ainsi N_c îles indexées de 1 à N_c.

4 Formulation MIP (Mixed Integer Programming)

Cette section détaille le modèle implémenté dans `M17.ipynb` avec **Pyomo**.

4.1 Variables de décision

Variable	Domaine	Interprétation
X_{ij}	$\{0, 1, 2\}$ (entier)	nombre de ponts entre les îles i et j
U_{ij}	$\{0, 1\}$ (binaire)	$U_{ij} = 1$ s'il existe au moins un pont entre i et j
flow_{ij}	réel dans $[0, 1]$	flot utilisé pour forcer la connexité
s	réel dans $[0, 1]$	source de flot associée à l'île de référence

4.2 Contrainte de degré (C1)

Pour chaque île c de valeur v_c, la somme des ponts incidents doit égaler v_c :

$$v_c = \sum_{j:(c,j)\in E} X_{cj} + \sum_{j:(j,c)\in E} X_{jc}. \tag{1}$$

C'est la traduction directe de la règle du chiffre sur l'île.

4.3 Contrainte de non-croisement (C2)

Deux segments candidats $[i, j]$ et $[m, n]$ ne peuvent pas être simultanément « ouverts » s'ils se croisent géométriquement. Dans le code, l'intersection est testée avec la bibliothèque `shapely` (`LineString`). Si croisement :

$$U_{ij} + U_{mn} \leq 1. \tag{2}$$

4.4 Couplage entre X et U (C3 et C3B)

On lie le nombre de ponts à l'indicateur binaire :

$$U_{ij} \leq X_{ij} \leq 2\,U_{ij}. \tag{3}$$

— si $U_{ij} = 0$, alors $X_{ij} = 0$ (pas de pont) ;

— si $U_{ij} = 1$, alors $X_{ij} \in \{1, 2\}$ (un ou deux ponts).

4.5 Contraintes de connexité par flot (C4A, C4B)

Pour garantir qu'il existe un chemin entre toutes les îles, le modèle utilise un mécanisme de **flot** :

— une île source (index 1 dans le code) injecte un petit flot ;

— chaque autre île « consomme » une petite quantité de flot ;

— le flot ne peut circuler que sur des liaisons ouvertes ($\text{flow}_{ij} \leq U_{ij}$ ou U_{ji}).

Intuition : si le graphe n'était pas connexe, le flot ne pourrait pas atteindre toutes les îles.

4.6 Fonction objectif

Le notebook minimise le nombre de couples d'îles reliées :

$$\min \sum_{(i,j)\in E} U_{ij}. \tag{4}$$

Sous les contraintes (1)–(3) et de connexité, on cherche donc une solution réalisable aussi « économe » que possible en nombre de liaisons utilisées.

4.7 Chaîne d'implémentation (notebook)

Le déroulement pratique dans `M17.ipynb` est :

1. installation / import de Pyomo ;
2. lecture de la grille CSV (`hashi*.csv`) ;
3. construction des îles et des arêtes candidates ;
4. création du modèle abstrait Pyomo (variables + contraintes) ;
5. résolution par un solveur (`cbc`, `glpk`, `ipopt`) ;
6. affichage de l'objectif et visualisation matplotlib des ponts.

Pour CBC, le notebook fixe notamment des options du type : `sec=600`, `threads=4`, `ratio=0.1`.

5 Depth First Search (DFS)

5.1 Principe

Le parcours en profondeur explore un graphe en allant aussi loin que possible le long d'une branche avant de revenir en arrière (*backtracking*). Sur un arbre de recherche :

— on part d'un nœud primaire ;

— on explore le premier fils jusqu'à une feuille ;

— on remonte, puis on explore le fils suivant, etc.

5.2 Intérêt pour Hashi

Des travaux antérieurs ont montré que le DFS peut accélérer la résolution de certains jeux de réflexion. Dans le notebook, une illustration NetworkX construit un graphe d'îles voisines et applique un DFS récursif à partir d'un nœud de départ, tout en visualisant le graphe.

5.3 Limites

Le DFS peut :

— s'engager durablement sur une mauvaise branche ;

— provoquer un backtracking massif ;

— gaspiller du temps de calcul sur de grandes instances.

C'est précisément pour éviter ces situations que le TIPE introduit le **Branch-and-Cut**, qui élimine des branches de l'arbre de recherche dès qu'elles ne peuvent plus contenir de solution optimale (ou réalisable intéressante).

6 Algorithme Branch-and-Cut

6.1 Idée générale

Le Branch-and-Cut couple deux mécanismes :

1. **Branchement** (*branch*) : on construit un arbre de recherche en fixant progressivement des variables entières (par exemple $U_{ij} = 0$ d'un côté, $U_{ij} = 1$ de l'autre).
2. **Coupes** (*cut*) : à chaque nœud, on résout une relaxation (souvent linéaire) enrichie d'inégalités supplémentaires qui resserrent l'ensemble réalisable.

Chaque nœud de l'arbre correspond donc à une relaxation résolue par une méthode de coupes ; l'arbre suit le principe générique du branchement.

6.2 Pourquoi c'est adapté à Hashi

Hashi est un problème combinatoire avec variables entières/binaires et contraintes linéaires (degré, non-croisement, liens X–U, flot). Un solveur MIP basé sur Branch-and-Cut peut :

— chercher une solution réalisable ;

— élaguer les branches infaisables ou non prometteuses ;

— fournir une valeur d'objectif et un temps d'exécution comparables d'une instance à l'autre.

6.3 Lien avec les solveurs

Dans la pratique du TIPE, on ne réécrit pas un Branch-and-Cut « à la main » pour chaque instance : on délègue cette stratégie aux solveurs MIP (en particulier CBC), tout en gardant une formulation mathématique explicite.

7 Solveurs utilisés

7.1 CBC

CBC (*COIN-OR Branch and Cut*) est un solveur MIP open source écrit en C++. Il met en œuvre un algorithme de Branch-and-Cut automatique. Dans le TIPE, c'est le solveur de référence pour la résolution entière.

7.2 GLPK

GLPK (*GNU Linear Programming Kit*) résout des programmes linéaires et des programmes en nombres entiers mixtes (MIP). Il sert de point de comparaison à CBC sur les mêmes instances.

7.3 Ipopt

Ipopt est conçu pour l'optimisation **non linéaire** sous contraintes (égalités et inégalités). Dans le TIPE, il est utilisé comme comparaison : ce n'est pas un solveur MIP classique, et les valeurs d'objectif obtenues ne sont donc pas interprétées de la même façon que celles de CBC/GLPK.

7.4 Protocole expérimental

Pour chaque instance :

1. on charge la même grille ;
2. on construit le même modèle Pyomo ;
3. on résout avec CBC, puis GLPK, puis Ipopt ;
4. on relève la **fonction objectif** et le **temps d'exécution**.

L'objectif est un **benchmarking** : comparer efficacité et comportement des solveurs.

8 Simulations et résultats

8.1 Instances étudiées

Trois exemples issus de la soutenance :

Instance	Dimension de grille	Nombre d'îles
Exemple 1	6×6	15
Exemple 2	12×12	48
Exemple 3	12×12	59

La soutenance présente également les figures de résolution correspondantes (solutions dessinées sur la grille pour chaque exemple).

8.2 Tableau de performances

Instance	Solveur	Fonction objectif	Temps
Exemple 1 (6×6, 15 îles)	CBC	14,0	0,04 s
	Ipopt	10,99	0,05 s
	GLPK	14,0	0,02 s
Exemple 2 (12×12, 48 îles)	CBC	**58,0**	**1,60 s**
	Ipopt	39,99	1,92 s
	GLPK	**58,0**	**80 s**
Exemple 3 (12×12, 59 îles)	CBC	47,0	0,78 s
	Ipopt	32,49	1,01 s
	GLPK	47,0	8,44 s

8.3 Analyse détaillée

Cohérence MIP (CBC vs GLPK). Sur les trois exemples, CBC et GLPK trouvent le **même** objectif entier : 14,0 (ex. 1), 58,0 (ex. 2), 47,0 (ex. 3). Cela confirme que, pour la formulation entière, les deux solveurs MIP convergent vers la même qualité de solution.

Écart de temps CBC / GLPK. L'écart le plus spectaculaire apparaît sur l'exemple 2 (48 îles) : CBC résout en **1,60 s**, GLPK en **80 s**, pour le même objectif 58,0. Le choix du solveur a donc un impact majeur sur le temps, même à solution MIP égale.

Sur l'exemple 3, CBC reste plus rapide (0,78 s contre 8,44 s). Sur l'exemple 1 (petite instance), GLPK est très rapide (0,02 s) ; l'écart est faible car le problème est petit.

Rôle d'Ipopt. Ipopt renvoie des objectifs différents : 10,99 ; 39,99 ; 32,49. Ces valeurs ne doivent pas être lues comme « meilleures » au sens MIP : Ipopt traite un problème d'optimisation non linéaire / continue, sans la même garantie d'intégralité. Dans le TIPE, Ipopt sert de **comparaison méthodologique**, pas de référence entière.

Lien avec la problématique. Les résultats illustrent concrètement l'intérêt d'une approche Branch-and-Cut / MIP : on obtient des solutions réalisables mesurables, et on quantifie l'efficacité (temps) selon le solveur et la taille d'instance.

9 Synthèse méthodologique

Le chemin suivi dans le TIPE peut se résumer ainsi :

1. **Problème** : Hashiwokakero (règles + connexité).
2. **Complexité** : NP-complet $\Rightarrow$ besoin d'outils d'optimisation.
3. **Modèle** : graphe $G = (V, E)$ puis MIP (X, U, flot, objectif $\sum U$).
4. **Baseline conceptuelle** : DFS (simple, mais limité).
5. **Méthode centrale** : Branch-and-Cut via solveurs MIP.
6. **Expériences** : CBC / GLPK / Ipopt sur 3 grilles.

7. **Conclusion expérimentale** : à objectif MIP égal, CBC domine nettement GLPK en temps sur les grandes instances testées.

10 Conclusion

Le TIPE montre qu'un puzzle NP-complet comme Hashiwokakero se prête naturellement à :

— une **modélisation en graphe** ;

— une **formulation MIP** (degré, non-croisement, connexité) ;

— une résolution par **Branch-and-Cut** (via CBC/GLPK) ;

— une **comparaison expérimentale** des solveurs.

Le DFS reste utile pédagogiquement pour comprendre la recherche dans un graphe, mais ses limites motivent le passage à l'optimisation. La comparaison CBC / GLPK / Ipopt met en évidence qu'au-delà de la formulation, le **choix du solveur** conditionne fortement le temps de calcul — notamment sur l'instance à 48 îles (1,60 s contre 80 s pour le même objectif 58,0).

11 Fichiers associés

Fichier	Rôle
`M17.ipynb`	Code Pyomo (MIP, DFS, solveurs, visualisation)
Soutenance (`.pptm`)	Diapositives du TIPE (hors aménagement urbain)
Fiche MCOT (`.pdf`)	Positionnement, problématique, bibliographie
Rapport LaTeX (`.tex`)	Sources de ce document
Rapport PDF (`.pdf`)	Version à lire / joindre au code

Références

[1] Wikipedia contributors, *Hashiwokakero*, 2020.

[2] L. Coelho, T. Vidal, *Benchmark Instances and Branch-and-Cut Algorithm for the Hashiwokakero Puzzle*, 2019.

[3] R. Malik, R. Efendi, E. Pratiwi, *Solving Hashiwokakero Puzzle Game with Hashi Solving Techniques and Depth First Search*, Bulletin of Electrical Engineering and Informatics, 2012.

[4] H. Lloyd, M. Crossley, M. Sinclair, M. Amos, *J-POP : Japanese Puzzles as Optimization Problems*, IEEE, 2022.

[5] S. Yen, S. Chiu, C. Chou, T. Su, *Hashi Solver*, The 15th Game Programming Workshop, 2010.

[6] Andersson, preuve de NP-complétude de Hashiwokakero, 2009 (citée dans le MCOT).

A Rappel des contraintes du modèle (vue code)

Pour faciliter la lecture croisée avec `M17.ipynb` :

Nom dans le code	Rôle
`C1`	égalité degré / chiffre de l'île
`C2`	non-croisement ($U_{ij} + U_{mn} \leq 1$)
`C3`	$X_{ij} \leq 2U_{ij}$
`C3B`	$X_{ij} \geq U_{ij}$
`C4A`	bilan de flot (source / consommation)
`C4B`	capacité de flot bornée par U
`obj1`	$\min \sum U_{ij}$